

PHẦN 1: GIỚI THIỆU CHUNG RASTERIZATION

GIỚI THIỆU RASTERIZATION

Rasterization là kỹ thuật chính được sử dụng trong các trò chơi video hiện đại để hiển thị hình ảnh trên màn hình

Vai trò: là kỹ thuật lựa chọn hàng đầu cho các ứng dụng **thời gian thực** trên PC, console và thiết bị di động

Có hai cách tạo hình ảnh trong đồ họa máy tính:

SOFTWARE RENDERING

Xử lý hoàn toàn trên CPU

Linh hoạt nhưng chậm

HARDWARE RENDERING

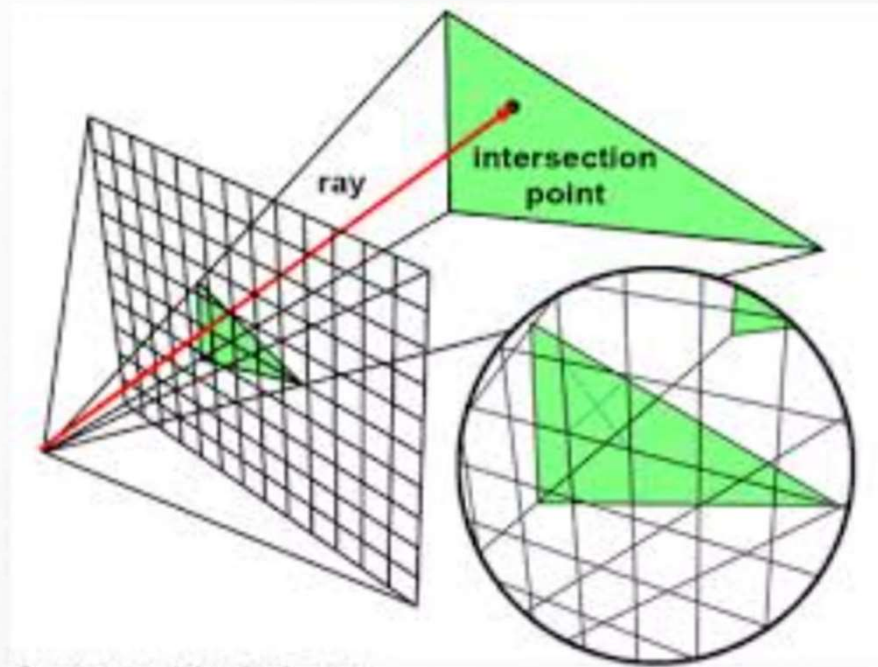
Xử lý bằng GPU (card đồ họa)

Nhanh hơn và tối ưu cho thời gian thực

❑ GPU được thiết kế chuyên biệt để xử lý đồ họa hiệu quả hơn CPU.

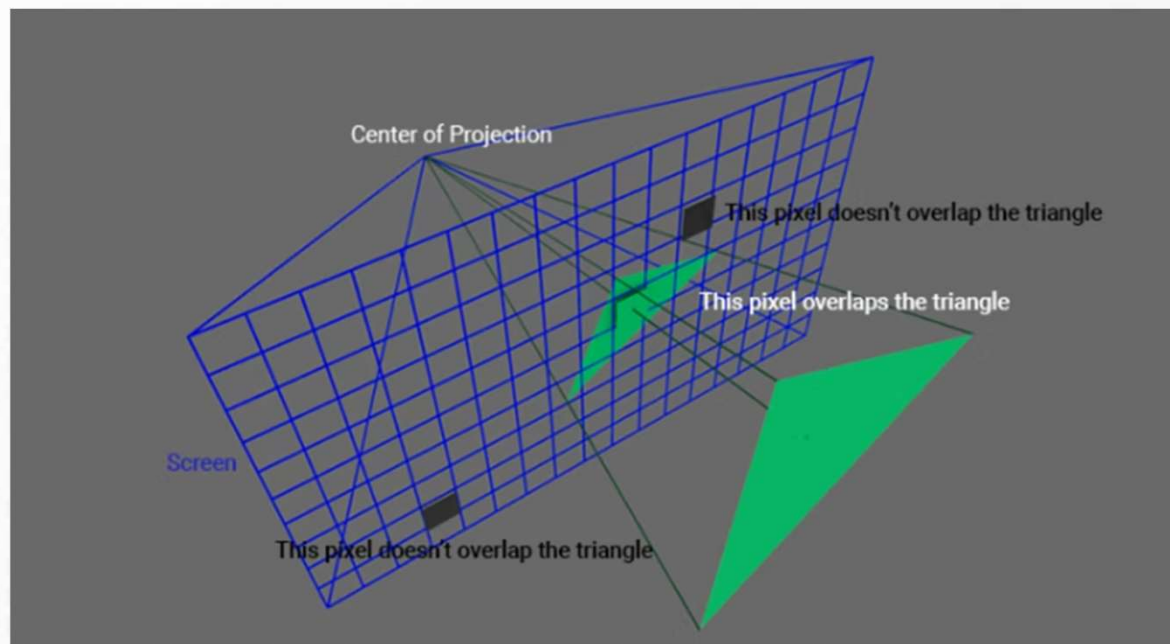
RASTERIZATION LÀ GÌ

Rasterization là quá trình chuyển đổi các thực thể hình học 3D (đa giác, tam giác) thành hình ảnh 2D trên màn hình.



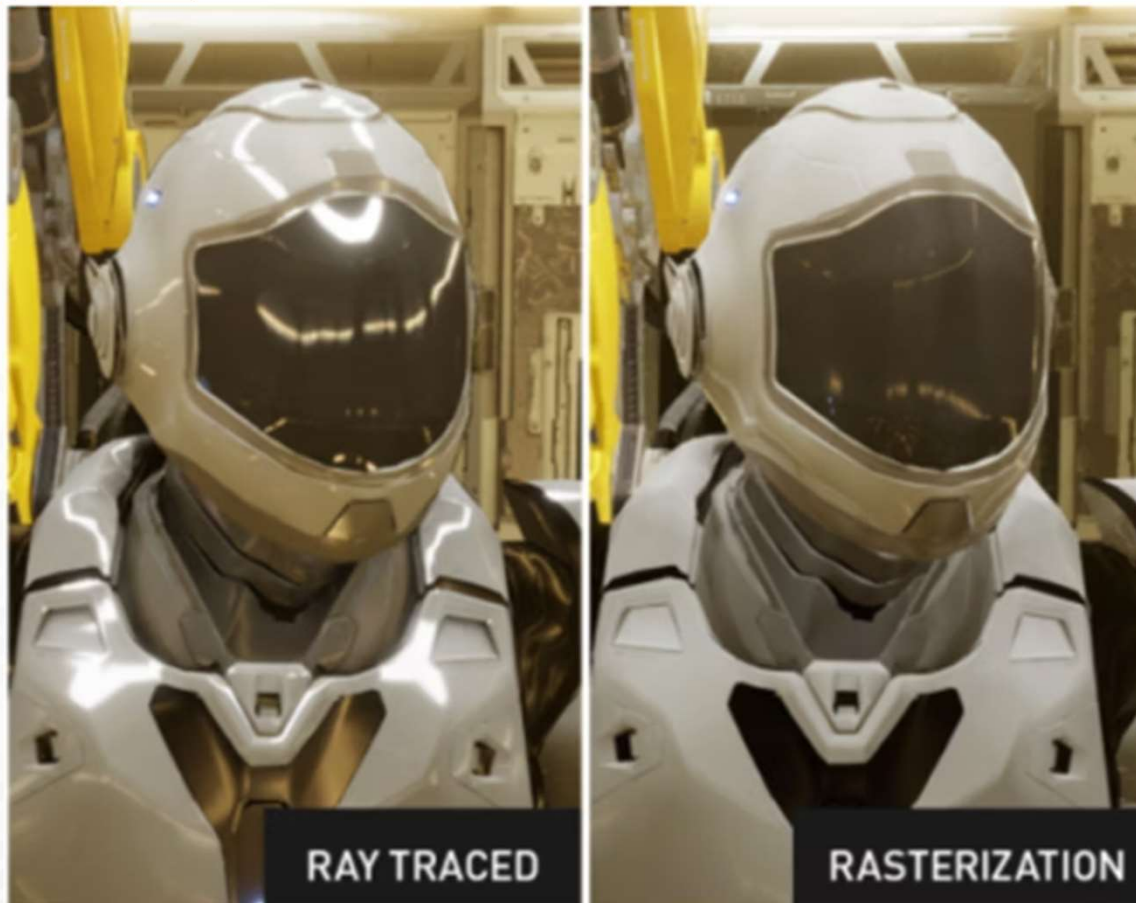
RASTERIZATION LÀ GÌ

Cơ chế: Đưa một luồng các tam giác qua phần cứng đồ họa, biến đổi chúng thành dạng 2D và hiển thị.



BẢNG SO SÁNH RASTERIZATION VS RAY TRACING

Đặc điểm	Rasterization	Ray Tracing
Quá trình	Chuyển đổi các vật 3D thành 2D bằng cách chiếu chúng lên màn hình, lấp đầy từng hàng điểm ảnh	Mô phỏng đường đi của tia sáng khi di chuyển qua cảnh, tính toán các điểm giao nhau với các vật thể để tạo ra ánh sáng, bóng đổ và phản chiếu chân thực
Tia trên mỗi điểm ảnh		Thông thường một tia cho mỗi điểm ảnh
Ứng dụng	Trong các ứng dụng thời gian thực như trò chơi điện tử	Ứng dụng trong các cảnh có độ chân thực cao như sản xuất phim và hiệu ứng hình ảnh cao cấp
Ánh sáng và đổ bóng	Độ chính xác hạn chế	Ánh sáng và phản xạ chính xác
Hiệu suất	Cực kỳ nhanh	Yêu cầu phần cứng mạnh mẽ
Yêu cầu phần cứng	Ít đòi hỏi hơn	Yêu cầu cao hơn, có thể chạy trên hầu hết các GPU hiện đại



Trade-off: Dù nhanh khó cài đặt hơn Ray Tracing và đôi khi phải hy sinh chất lượng hình ảnh để đổi lấy hiệu suất

SOFTWARE RENDERING VS HARDWARE RENDERING

Tại sao phải học dựng hình trên CPU? Để hiểu các thuật toán nền tảng trước khi sử dụng API đồ họa.

SOFTWARE RENDERING

Tất cả tính toán được thực hiện trên CPU. Chậm hơn nhưng giúp hiểu nguyên lý cơ bản.

Thiết bị chuyên dụng luôn hiệu quả hơn bộ xử lý đa năng khi thực hiện các tác vụ chuyên biệt.

HARDWARE RENDERING

CPU chuyển giao dựng hình cho GPU, giải phóng CPU cho AI, vật lý và các tác vụ khác. GPU được tối ưu hóa cho **Rasterization** nên nhanh hơn nhiều.

PHẦN 2: KỸ THUẬT RASTERIZATION

KHÁI NIỆM RASTERIZATION

Rasterization là quá trình chuyển đổi tam giác 3D thành pixel 2D trên màn hình.



=> Kết quả là hình ảnh hoàn chỉnh trên màn hình.

QUY TRÌNH CHUNG RASTERIZATION

CLIPPING

- Loại bỏ các tam giác cùng hướng
 - Xác định Bounding Box bao quanh tam giác
- Giảm số pixel cần kiểm tra

PIXEL TEST

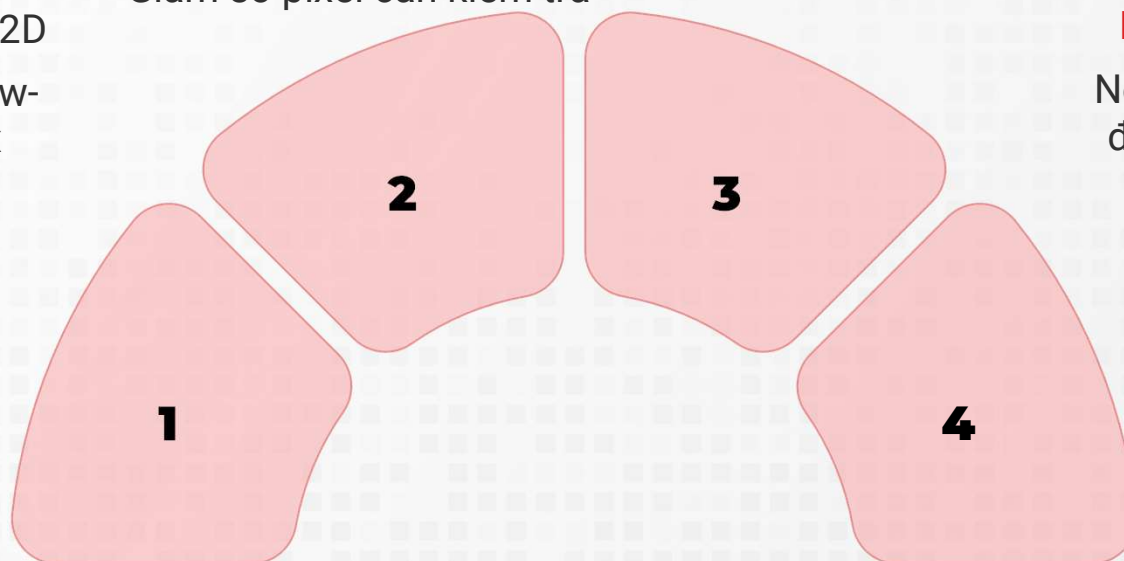
Kiểm tra pixel có nằm trong tam giác
Sử dụng thuật toán Barycentric

INTERPOLATION

Nội suy các giá trị pixel được xác định là nằm trong: Color, Depth, Texture

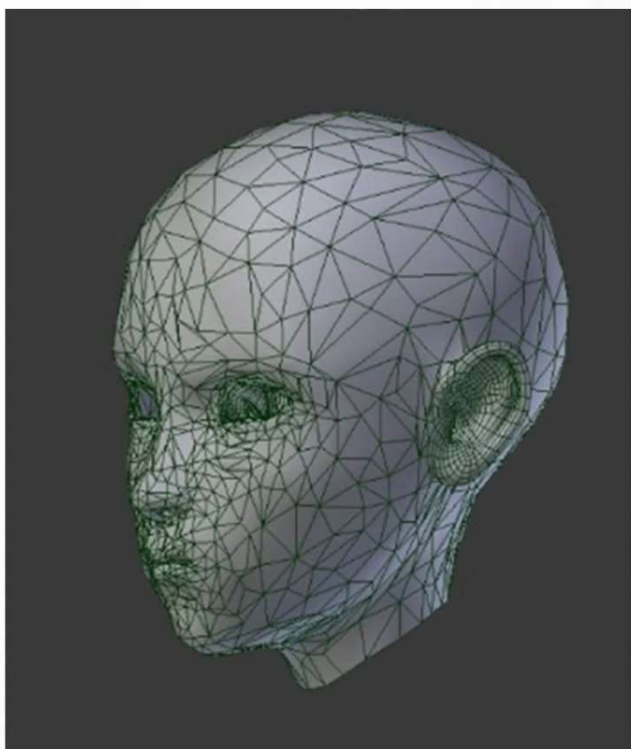
TRANSFORM

Chuyển từ 3D sang 2D
Sử dụng Model-View-Projection Matrix



VẬT THỂ TRONG ĐỒ HỌA 3D

Mọi vật thể 3D đều được tạo thành từ tam giác.



TẠI SAO DÙNG TAM GIÁC?

- Là đa giác đơn giản nhất
- Luôn phẳng
- Dễ tính toán
- Tối ưu cho GPU

✓ Tam giác là đơn vị cơ bản của đồ họa 3D.

VÍ DỤ VẬT THỂ

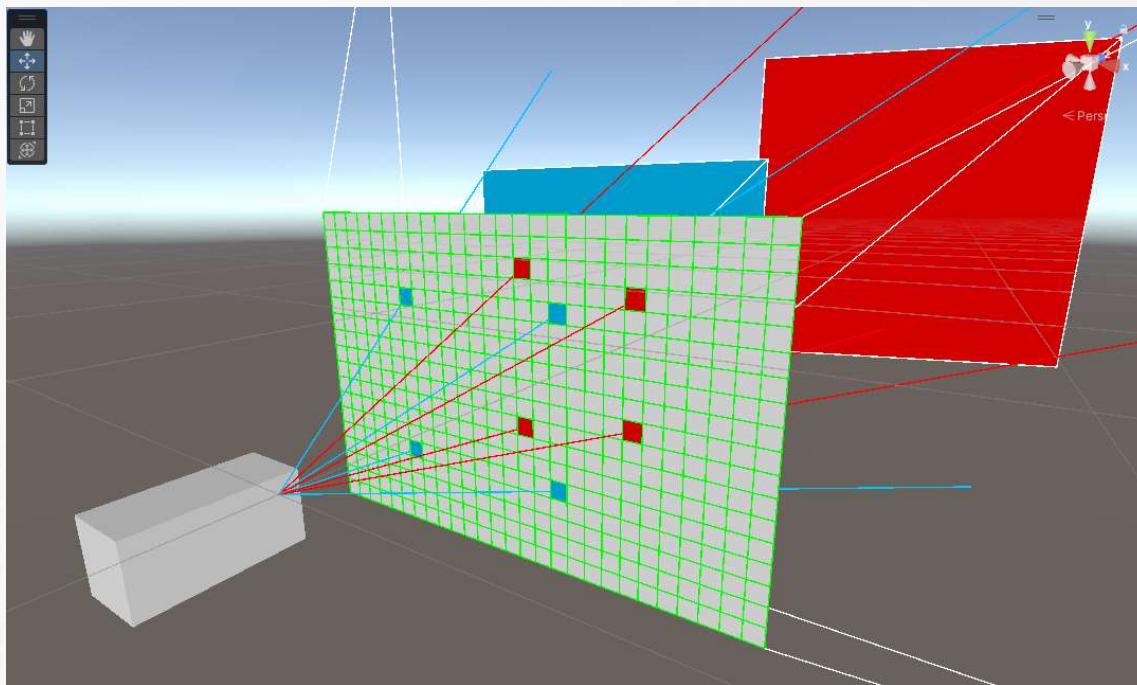
- Nhân vật
- Xe
- Toà nhà

Transformation 3D to 2D

Sử dụng kỹ thuật Model View Projection Matrix để tiến hành chuyển đổi một điểm từ không gian 3D sang mặt phẳng 2D

Thành phần của Model View Projection Matrix

- Ma trận đối tượng (Model Matrix)
- Ma trận góc nhìn (View Matrix)
- Ma trận chiếu (Projection matrix)
 - Perspective Projection
 - Orthographic Projection



Tham khảo: https://www.codinglabs.net/article_world_view_projection_matrix.aspx

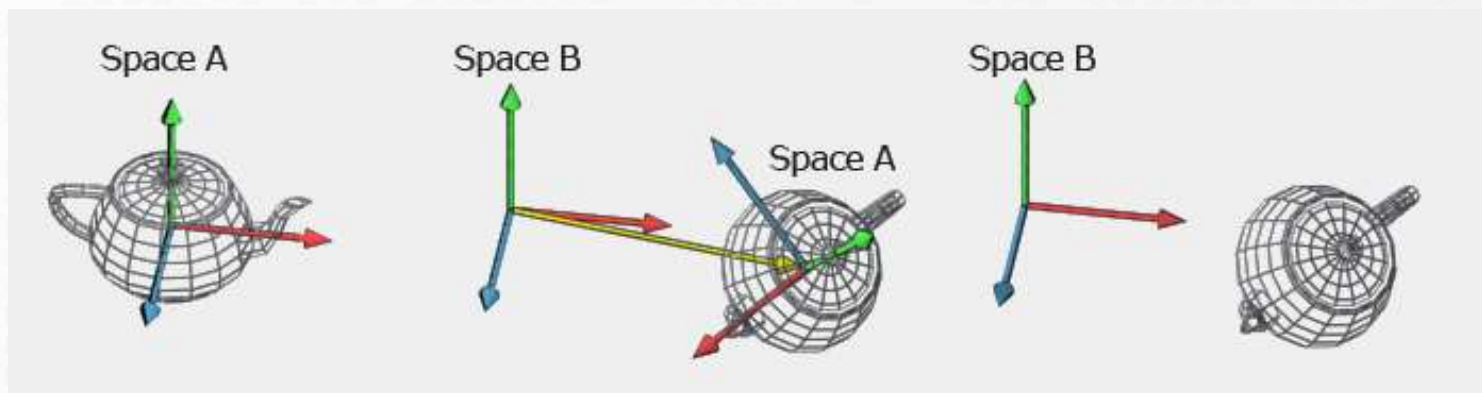
Phép chuyển đổi hệ tọa độ không gian

Chuyển đổi tọa độ của đối tượng từ hệ tọa độ A sang B

Một hệ trục tọa độ được biểu diễn bằng:

- 3 Vector Ox , Oy và Oz (có hướng biểu diễn độ xoay và độ dài thể hiện tỉ lệ)
- Tâm hệ tọa độ

Thông qua áp dụng các phép biến đổi **Affine**: Tịnh tiến (Translation), Xoay (Rotation) và Tỉ lệ (Scale)



Phép biến đổi Affine

Sử dụng để thực hiện các phép biến đổi trục tọa độ

Unity: Matrix4x4.TRS(translation, rotation, scale)

$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$	$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$Sh = \begin{bmatrix} 1 & sh_x^y & sh_x^z & 0 \\ sh_y^x & 1 & sh_y^z & 0 \\ sh_z^x & sh_z^y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Translation Matrix	Scaling Matrix	Shear Matrix
$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Rotation Matrix		

Ma trận đối tượng (Model Matrix)

Chuyển đổi tọa độ các đỉnh của tam giác từ Model Space (Local Space) sang World Space

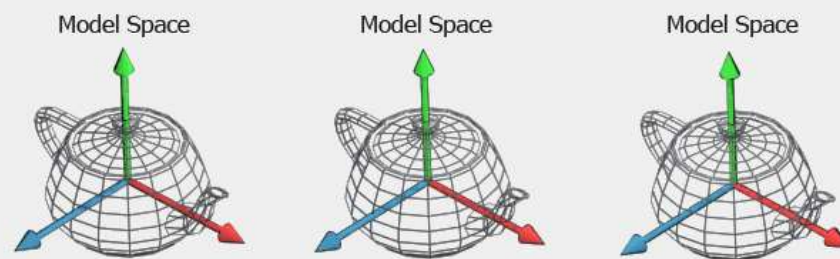


Figure 6: Three teapots each one in its own model space

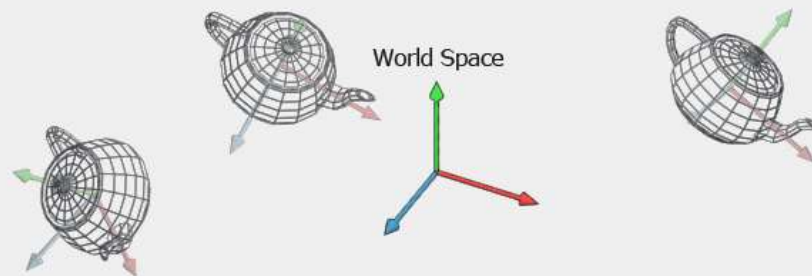
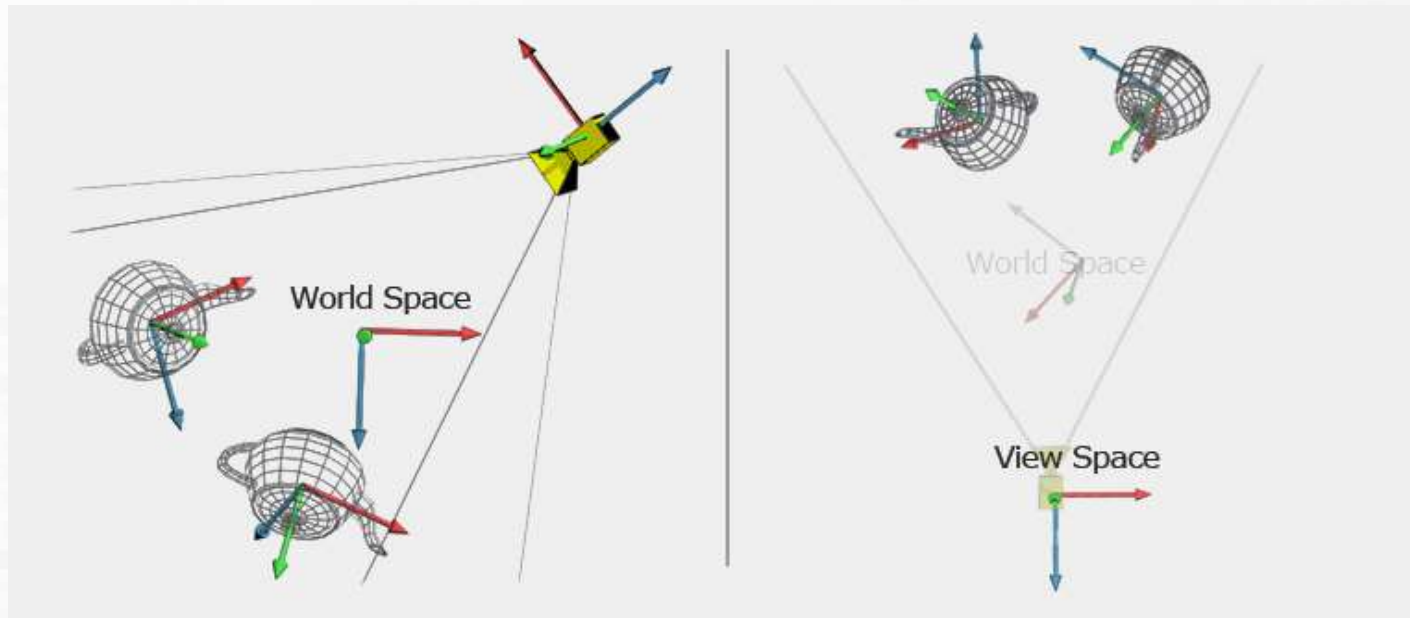


Figure 7: Three teapots set in World Space

Ma trận góc nhìn (View Matrix)

Chuyển đổi tọa độ từ World Space sang Local Space của Camera

$\text{ViewMatrix} = \text{CameraModelMatrix}$ nghịch đảo

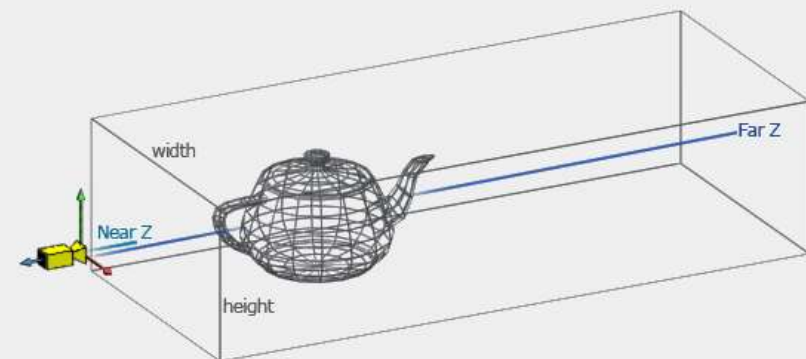
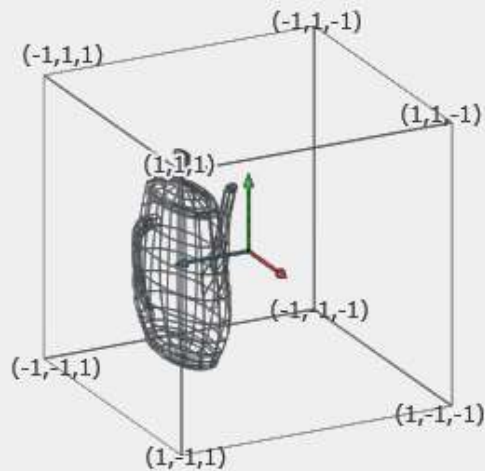


ORTHOGRAPHIC PROJECTION MATRIX

$$\begin{bmatrix} \frac{1}{width} & 0 & 0 & 0 \\ 0 & \frac{1}{height} & 0 & 0 \\ 0 & 0 & \frac{2}{Z_{far} - Z_{near}} & -\frac{Z_{far} + Z_{near}}{Z_{far} - Z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Trong đó:

- **width** là độ dài chiều ngang màn hình
- **height** là độ dài chiều dọc màn hình
- **Znear** là khoảng cách gần nhất nhìn thấy
- **Zfar** là khoảng cách xa nhất nhìn thấy



PERSPECTIVE PROJECTION MATRIX

$$\begin{bmatrix} \tan^{-1}\left(\frac{FOV_x}{2}\right) & 0 & 0 & 0 \\ 0 & \tan^{-1}\left(\frac{FOV_y}{2}\right) & 0 & 0 \\ 0 & 0 & -\frac{Z_{far} + Z_{near}}{Z_{far} - Z_{near}} & -\frac{2(Z_{near} Z_{far})}{Z_{far} - Z_{near}} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Trong đó:

- FOV_x là phạm vi góc nhìn theo chiều ngang
- FOV_y là phạm vi góc nhìn theo chiều dọc
- Z_{near} là khoảng cách gần nhất nhìn thấy
- Z_{far} là khoảng cách xa nhất nhìn thấy

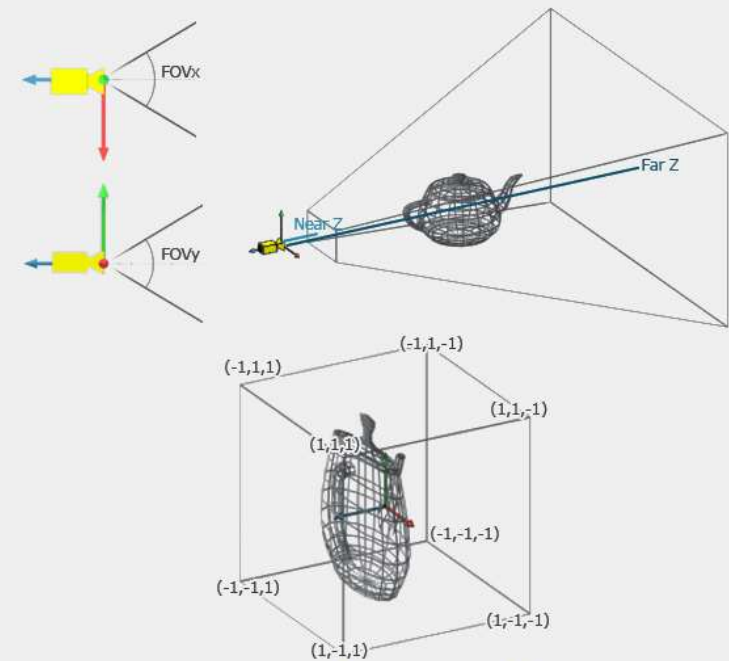


Figure 11: Perspective Projection

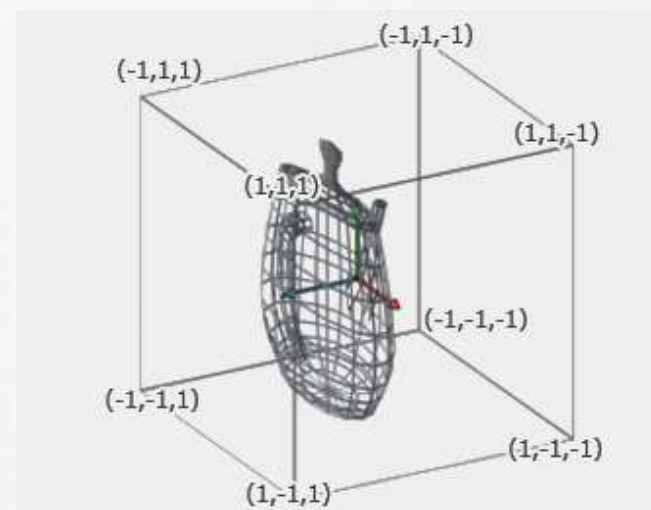
Kết quả Model View Projection

Thu được một điểm có tọa độ (x, y, z, w)

Cần chuẩn hóa tọa độ (x, y, z, w) về dạng $(x/w, y/w, z/w, 1)$

Tọa độ $(x, y, z, 1)$ thu được từ phép biến đổi MVP:

- x chạy từ -1 đến 1 tương ứng với biên trái đến biên phải
- y chạy từ -1 đến 1 tương ứng với biên trái đến biên phải
- z chạy từ -1 đến 1 tương ứng với biên trái đến biên phải



https://en.wikipedia.org/wiki/Homogeneous_coordinates

CLIPPING

Clipping loại bỏ phần không nhìn thấy:

- Nằm ngoài phạm vi camera
- Tam giác cùng hướng với camera
- Xác định Bounding Box



Chỉ phần nằm trong viewport mới được render.

LỢI ÍCH



Tăng hiệu suất



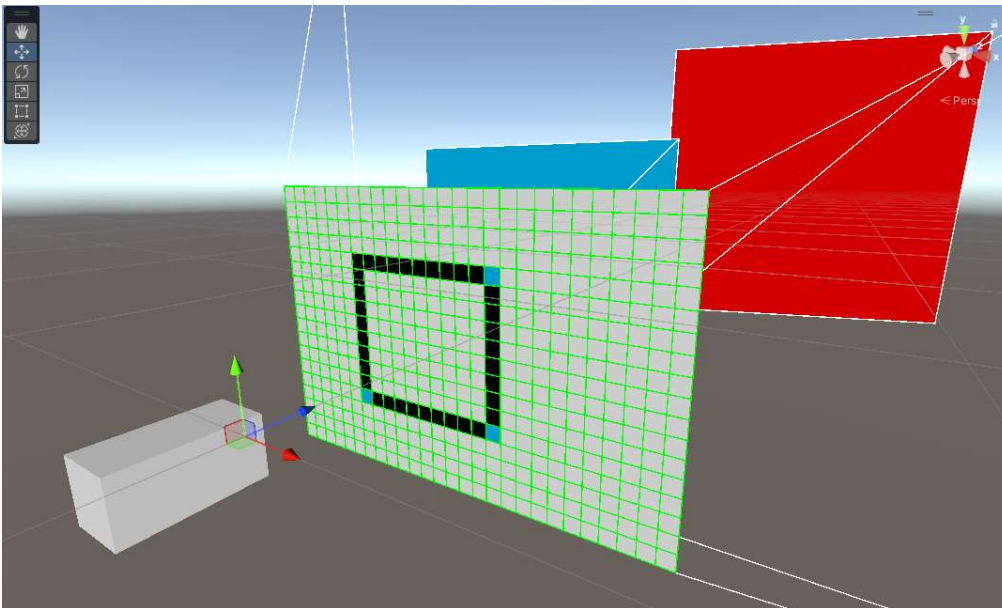
Giảm tính toán



Tránh lỗi bộ nhớ

CLIPPING

Clipping loại bỏ phần không nhìn thấy.



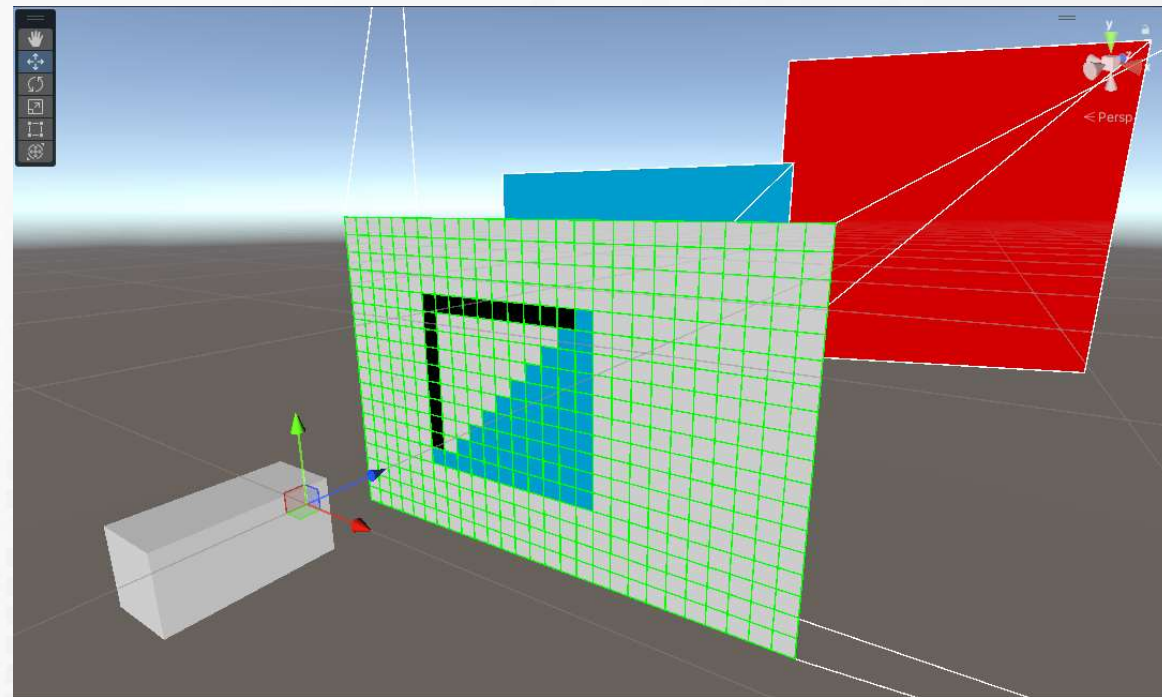
Xác định bounding box trong Viewport:

- minX, maxX
- minY, maxY
- Giới hạn trong phạm vi của cửa sổ

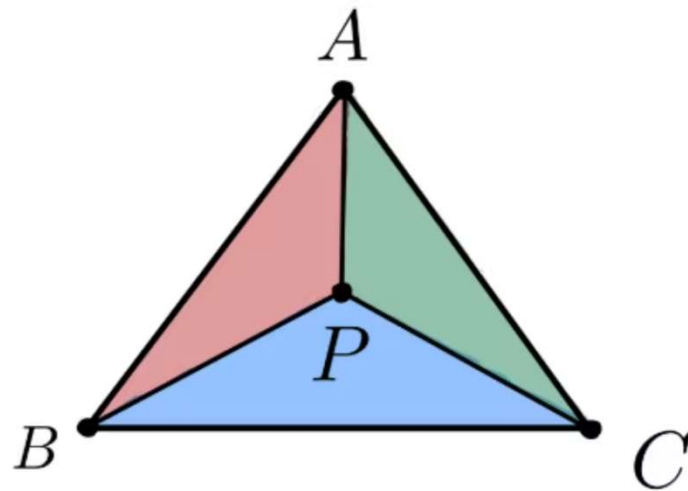
☑ Chỉ phần nằm trong bounding box mới được render.

PIXEL TEST

1. Tính toán tọa độ Barycentric Coordinate (α , β , γ) của pixel
2. Kiểm tra: $\alpha + \beta + \gamma = 1$
3. Nếu tổng xấp xỉ 1, pixel nằm trong tam giác



BARYCENTRIC COORDINATE



$$\alpha = \frac{\text{Area}(\triangle PBC)}{\text{Area}(\triangle ABC)}$$

$$\beta = \frac{\text{Area}(\triangle PAC)}{\text{Area}(\triangle ABC)}$$

$$\gamma = \frac{\text{Area}(\triangle PAB)}{\text{Area}(\triangle ABC)}$$

$$P = \alpha A + \beta B + \gamma C$$

INTERPOLATION

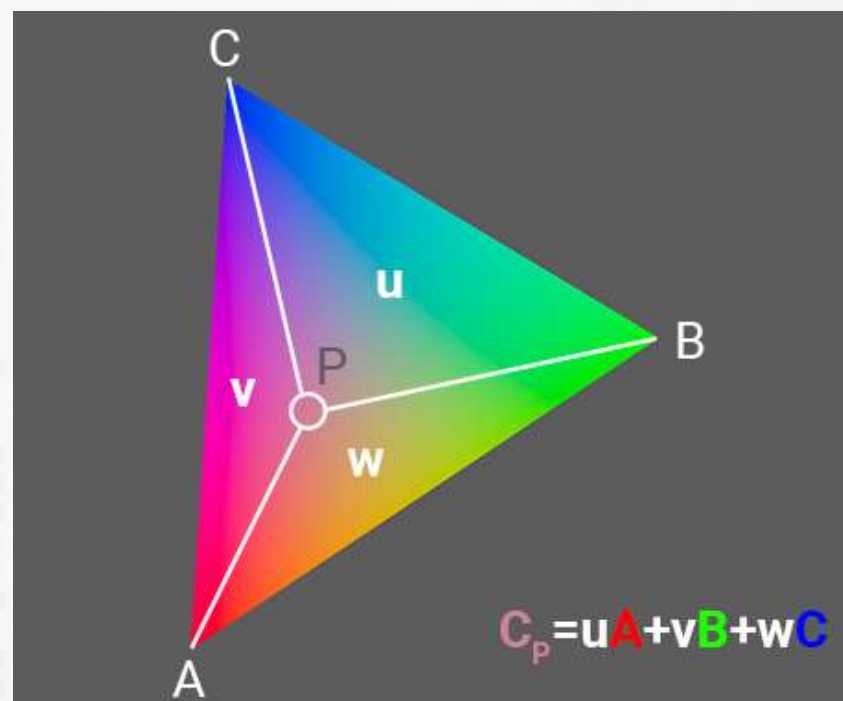
Nội suy các thông tin như màu sắc, độ sâu, vector pháp tuyến, ... của các điểm bên trong tam giác dựa trên thông tin của các đỉnh và tọa độ Barycentric coordinate

XÁC ĐỊNH ĐỘ SÂU TẠI PIXEL

Xác định độ sâu tại từng pixel để sắp xếp.

TÔ MÀU PIXEL

Nội suy màu sắc của pixel dựa trên màu sắc của 3 đỉnh.



PHẦN 3: RENDERING BUFFERS

RENDERING BUFFERS

- **SỬ DỤNG XUYÊN SUỐT QUÁ TRÌNH TẠO CẢNH**

- **MẢNG BỘ NHỚ DÙNG ĐỂ GHI VÀ ĐỌC THÔNG TIN KHUNG HÌNH**

- ☑ Giúp GPU quản lý rendering hiệu quả.

SECONDARY BUFFERS

Secondary Buffers sử dụng 2 buffer

FRONT BUFFER

Đang hiển thị trên màn hình

BACK BUFFER

Đang render hình ảnh tiếp theo

LỢI ÍCH

TRÁNH FLICKERING

TRÁNH TEARING

HIỂN THỊ MƯỢT HƠN

Z-BUFFER

Dùng để:

- Xác định pixel gần camera hơn
- Quyết định pixel nào hiển thị

Giúp hiển thị đúng thứ tự vật thể

